Activation Functions- Types

Thursday, September 10, 2026 7:01 PM

6. Activation Functions

This is one of the **most important deep-learning interview topics**.

An activation function transforms the weighted sum:

$$a = f(z)$$

Why do we need it?

Without activation functions

Suppose:

$$z_1 = W_1 X$$

and:

$$z_2 = W_2 z_1$$

Then:

$$z_2 = W_2 W_1 X$$

This is still just a linear transformation.

Even if you stack 100 linear layers, the overall operation can still be represented as one linear transformation.

Therefore:

Activation functions introduce non-linearity.

This allows neural networks to learn complex relationships.

6a. Sigmoid

Definition

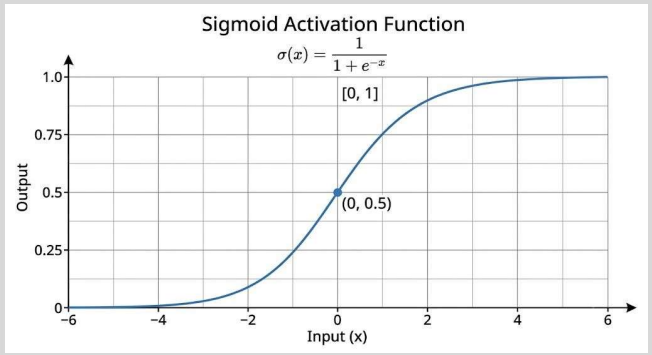
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

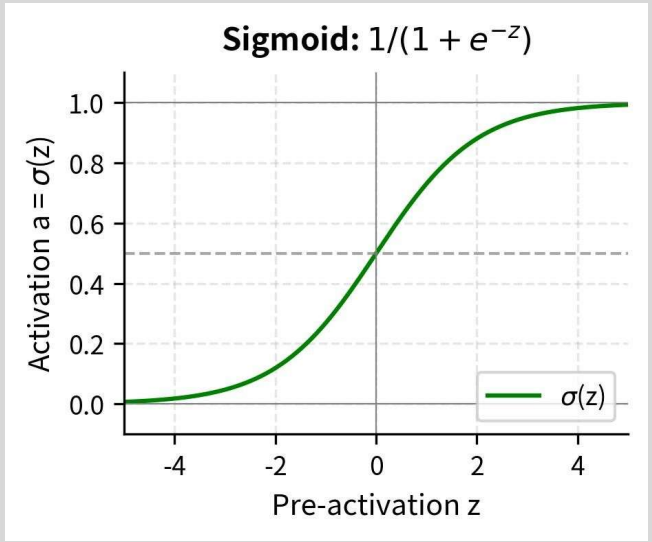
Range:

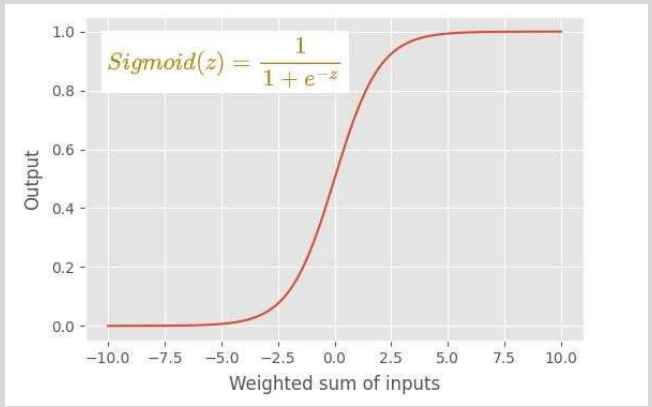
$$0 < \sigma(x) < 1$$

Shape:

Output
```
 1 |            ______
   |          __/
   |        __/
0.5 |---------/
   |     __
   |   __/
 0 |___/
   +---------------------- Input
```







4

Important properties

For:

$$x \to +\infty$$

$$\sigma(x) \to 1$$

For:

$$x \to -\infty$$

$$\sigma(x) \to 0$$

At:

$$x = 0$$

$$\sigma(0) = 0.5$$

Derivative

$$\sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big)$$

Maximum derivative is only:

$$0.25$$

This contributes to the **vanishing-gradient problem** in deep networks.

Use cases

Commonly used for:

- Binary classification output
- Probability of a binary event
- Logistic regression
- Binary neural-network output layers

For example:

CHD = 1
No CHD = 0

Sigmoid output = 0.83

Interpretation:

Estimated probability ≈ 83%

Problems

- Vanishing gradients

- Saturation
- Not zero-centered
- Usually not preferred for hidden layers of modern deep networks

Interview question

Why isn't sigmoid commonly used in hidden layers?

Because for very positive or negative inputs, its derivative approaches zero, causing gradients to become very small.

6b. tanh()

Definition

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

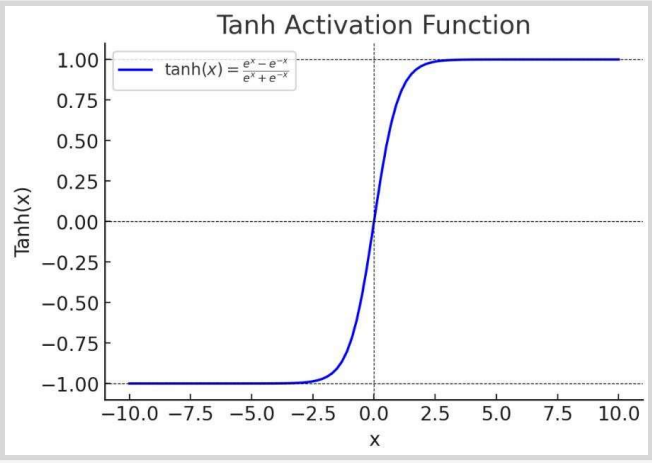
Range:

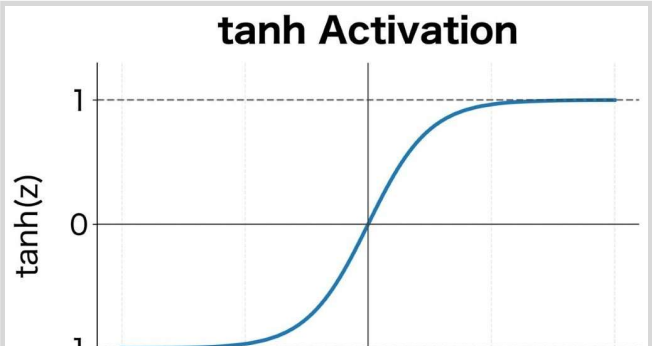
$$-1 < \tanh(x) < 1$$

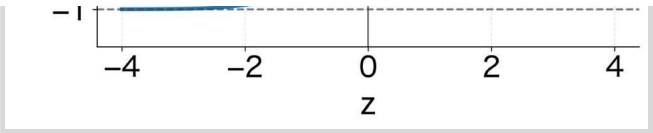
Shape:

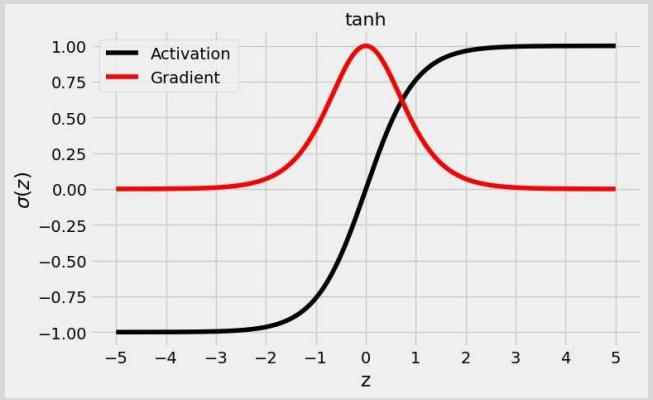
Output

```
1 |          ______
  |        __/
  |      __/
 0 |-------/
  |   __
  | __/
-1 |______
  +---------------- Input
```









4

Important property

Unlike sigmoid:

Sigmoid → 0 to 1

tanh → -1 to +1

Therefore tanh is **zero-centered**.

Derivative

$$\tanh'(x) = 1 - \tanh^2(x)$$

Advantages

- Zero-centered
- Stronger gradients around zero than sigmoid
- Historically popular in hidden layers

Disadvantage

It can still suffer from:

Vanishing gradients

because it saturates at -1 and +1.

Use cases

Historically:

- Hidden layers
- RNNs

Modern networks generally favor ReLU-family functions for many hidden layers.

6c. ReLU

Definition

ReLU = **Rectified Linear Unit**

$$ReLU(x) = \max(0, x)$$

Therefore:

$$ReLU(x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0 \end{cases}$$

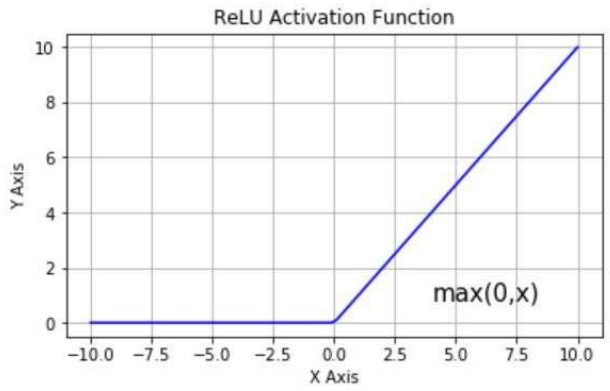
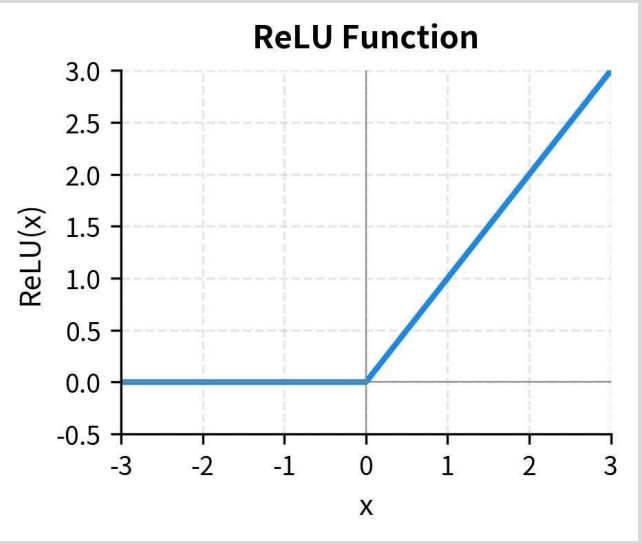
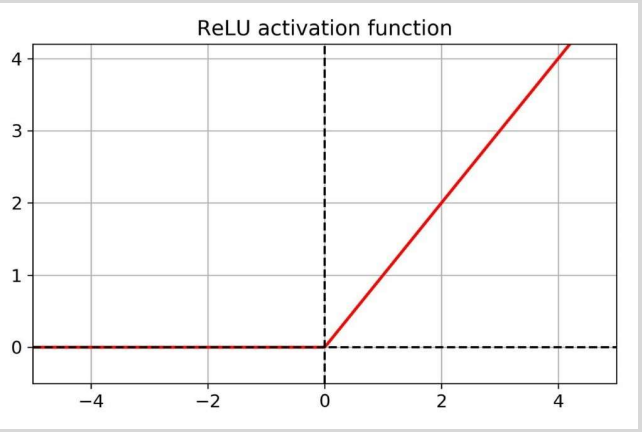


Figure: ReLU Activation Function





4

Shape

Output

```
  |
  |      /
  |     /
  |    /
  |___/_____________
  |
```

+---------- Input
0

Why ReLU became so popular

For positive values:

$$ReLU(x) = x$$

and:

$$ReLU'(x) = 1$$

Therefore it doesn't saturate on the positive side.

This helps with the vanishing-gradient problem compared with sigmoid/tanh.

Advantages

- Computationally simple
- Fast
- Sparse activations
- Good gradient for positive values
- Excellent default for many hidden layers

Problem: Dying ReLU

For:

$$x < 0$$

the output is zero and derivative is zero.

A neuron can potentially become permanently inactive.

This is called:

Dying ReLU

Use cases

Very common in:

- CNNs
- MLPs
- Deep neural networks

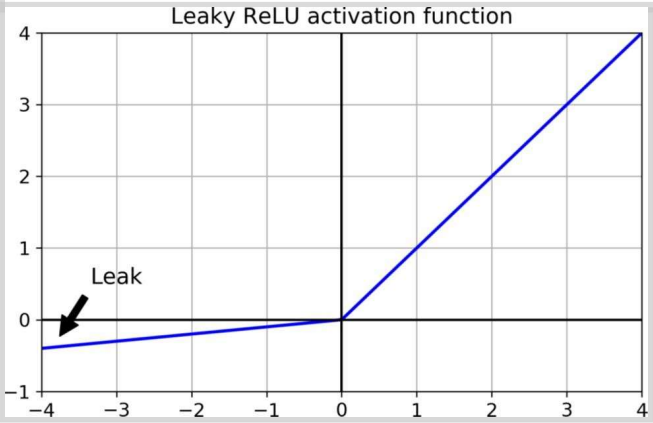
6d. Leaky ReLU

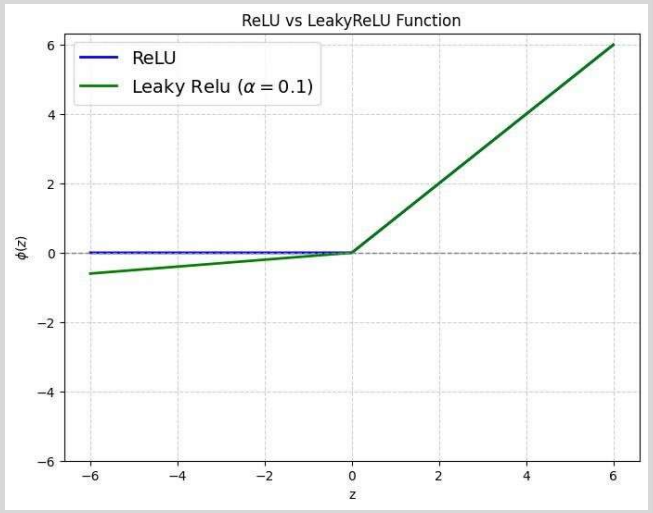
Leaky ReLU modifies ReLU by allowing a small negative slope.

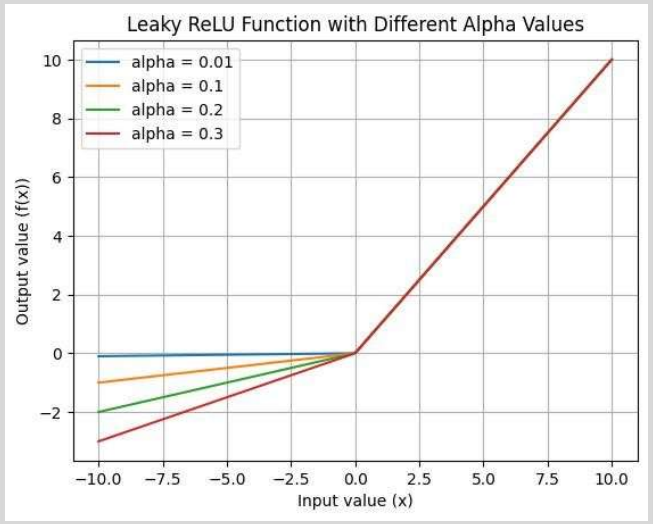
$$f(x) = \begin{cases} \alpha x & x < 0 \\ x & x \geq 0 \end{cases}$$

Usually:

$\alpha = 0.01$







4

Instead of:

Negative → 0

we get:

Negative → small negative value

Example:

$x = -10$

With ReLU:

$$0$$

With Leaky ReLU, α = 0.01:

$$-0.1$$

Main advantage

It reduces the risk of dead neurons because the negative side still has a gradient.

Use case

Useful when:

- ReLU neurons are dying
- Negative activations need to retain some gradient

Interview question

ReLU vs Leaky ReLU?

ReLU:
x < 0 → 0

Leaky ReLU:
x < 0 → αx

6e. Maxout

Maxout is a more flexible activation function.

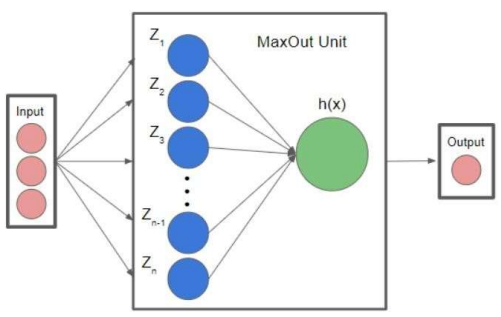
Instead of having a fixed mathematical curve like ReLU, Maxout learns multiple linear functions and selects the maximum.

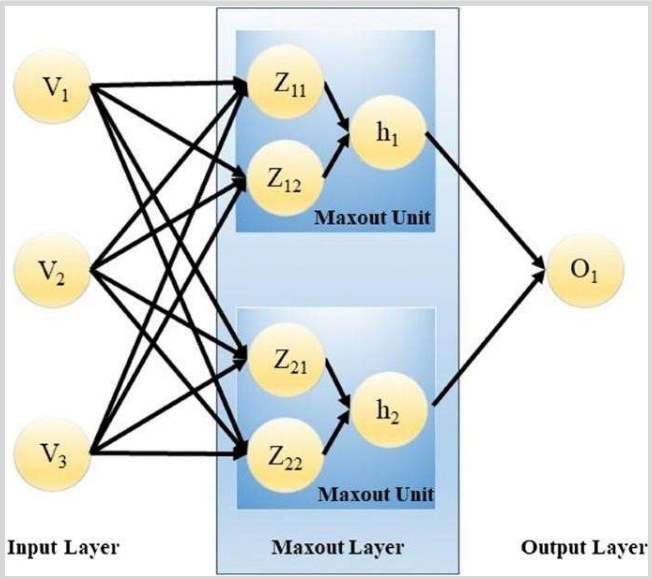
For example:

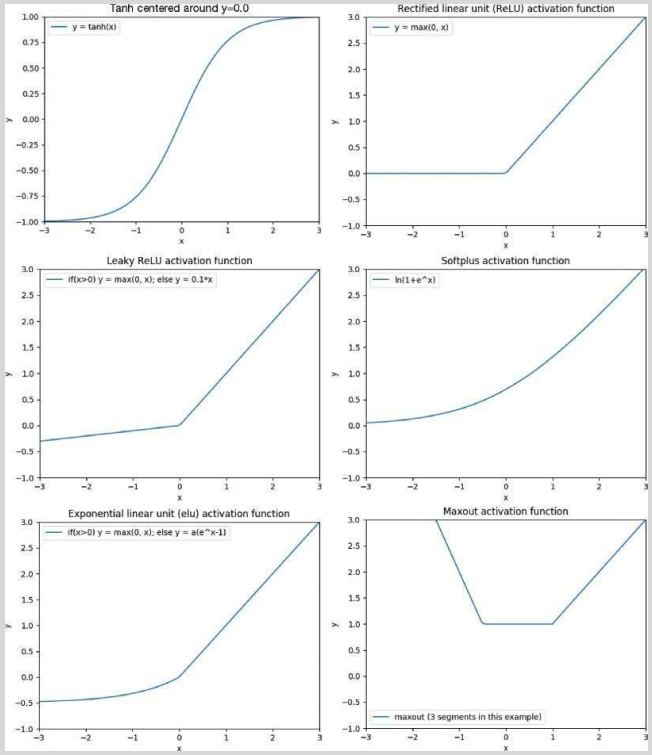
$$f(x) = \max(w_1^T, x + b_1 \; w_2^T x + b_2)$$

More generally:

$$f(x) = \max_{i=1,\ldots,k} \left(w_i^T x + b_i\right)$$







4

Intuition

Imagine several candidate lines:

Line 1 ———／
Line 2 ——／—
Line 3 —／——

↓

Maxout chooses
the highest value
at each x

Therefore Maxout can represent a **piecewise-linear function**.

Advantages

- Flexible
- Can learn activation shape
- Can avoid the standard dying-ReLU behavior
- Generalizes ReLU-like behavior

Disadvantage

It requires more parameters and computation than ReLU.

H2O's deep-learning documentation notes that Maxout is computationally more expensive but can sometimes improve accuracy.

Interview point

Maxout is not simply:

> "Take the maximum of neurons."

It is:

> **Take the maximum of several learned affine functions.**

That distinction is important.

6f. Softmax

Softmax is primarily used for **multi-class classification output**.

Suppose we have:

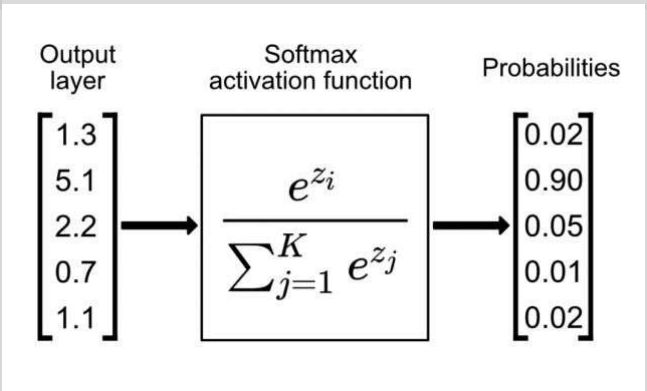
Cat
Dog
Horse

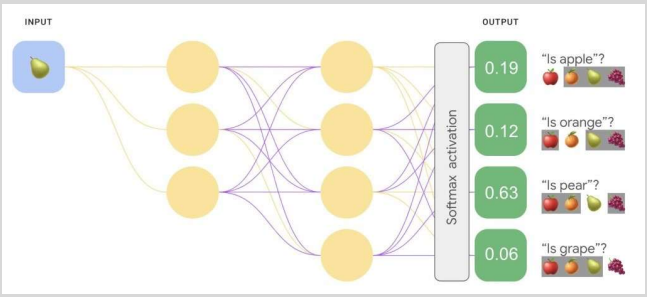
The network produces logits:

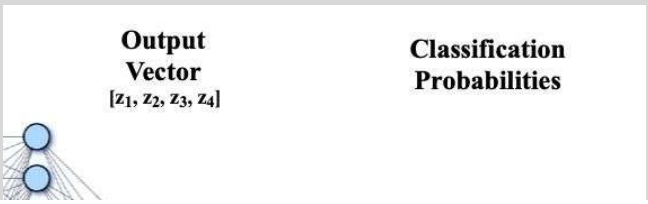
Cat = 2.5
Dog = 1.2
Horse = 0.3

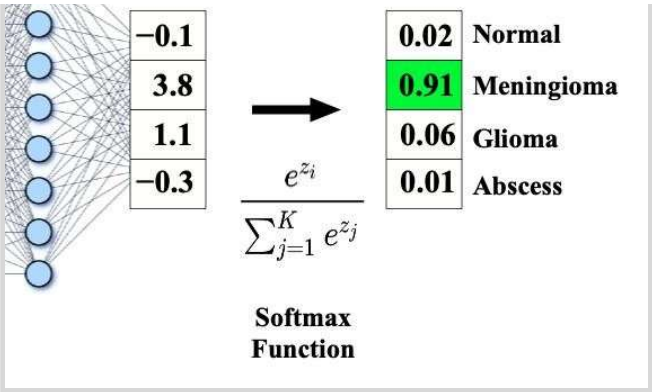
Softmax converts them into probabilities.

$$P(y=i)=\frac{e^{z_i}}{\sum_j e^{z_j}}$$









6

Example:

Cat 0.72
Dog 0.20
Horse 0.08

1.00

The probabilities sum to 1:

$$\sum_i P_i = 1$$

Use case

Multi-class classification where classes are mutually exclusive:

Image → Cat / Dog / Horse

Softmax vs Sigmoid

This is a very common interview question.

Binary classification:

Sigmoid
↓
Probability of class 1

Multi-class mutually exclusive classification:

Softmax
↓
P(class 1), P(class 2), P(class 3)...

Important distinction

For **multi-label classification**, where several labels can simultaneously be true, sigmoid is usually used independently for each label rather than softmax.

7. Activation Function — Interview Comparison

Function	Range	Main use	Major problem
Sigmoid	0 to 1	Binary output	Vanishing gradient
tanh	-1 to 1	Older hidden layers/RNNs	Vanishing gradient
ReLU	0 to ∞	Hidden layers	Dying ReLU
Leaky ReLU	-∞ to ∞	Hidden layers	Hyperparameter α
Maxout	-∞ to ∞	Specialized hidden layers	More parameters
Softmax	0 to 1, sum = 1	Multi-class output	Can become numerically sensitive

A useful modern rule of thumb:

Hidden layer
↓
ReLU / Leaky ReLU / other modern variants

Binary output
↓
Sigmoid

Multi-class output
↓
Softmax